Exemple Détaillé : Traitement d'une Phrase dans le Pipeline — T-AIA-911

Phrase d'exemple : *"J'aimerais prendre le train de Bordeaux pour rejoindre Strasbourg"*

Étape 0 : Entrée Utilisateur

L'utilisateur soumet la phrase via :

- L'interface chat du frontend (POST vers `/predict`)
- Un fichier CSV (POST vers `/process_csv`) au format `sentenceID,sentence`

```
Entrée brute: "J'aimerais prendre le train de Bordeaux pour rejoindre Strasbourg"
```

Étape 1 : CLASSIFIER — Validation de l'Intention

1.1 Tokenisation

Le tokenizer CamemBERT découpe la phrase en sous-tokens :

```
Texte    : J'   aimerais   prendre   le    train   de   Bordeaux   pour   rejoindre   Strasbourg
Tokens   : _J'  _aimerais  _prendre  _le   _train  _de  _Bordeaux  _pour  _rejoindre  _Strasbourg
IDs      : [5, 10429, 14832, 83, 2189, 65, 12304, 284, 29183, 44921]
```

Le tokenizer ajoute les tokens spéciaux :

```
[CLS] + tokens + [SEP] + [PAD] × (128 − len)
```

1.2 Passage dans CamemBERT

- **12 couches d'attention bidirectionnelle** analysent le contexte global
- Le token `[CLS]` capture la représentation globale de la phrase
- Sortie : vecteur de 768 dimensions pour `[CLS]`

1.3 Classification

```
[CLS] embedding (768-d)
    │
    ▼
Linear(768 → 2)
    │
    ▼
logits: [INVALID=-3.2, VALID=4.1]
    │
    ▼
Softmax: P(INVALID)=0.027, P(VALID)=0.973
    │
    ▼
Décision: ✅ VALID (confiance 97.3%)
```

→ La phrase passe au module NER.

Étape 2 : NER — Extraction des Entités

2.1 Tokenisation avec Offset Mapping

Le tokenizer retourne la position caractère par caractère de chaque token :

```
Token           | Début | Fin | Prediction
----------------+-------+-----+----------
[CLS]           |   0   |  0  | O
_J'             |   0   |  2  | O
_aimerais       |   2   | 11  | O
_prendre        |  12   | 19  | O
_le             |  20   | 22  | O
_train          |  23   | 28  | O
_de             |  29   | 31  | O
_Bordeaux       |  32   | 40  | B-DEPARTURE ← début entité départ
_pour           |  41   | 45  | O
_rejoindre      |  46   | 55  | O
_Strasbourg     |  56   | 66  | B-ARRIVAL   ← début entité arrivée
[SEP]           |   0   |  0  | O
```

2.2 Passage dans CamemBERT NER

Chaque token reçoit sa propre classification :

```
Per-token logits (128 × 5):
    token[6] "Bordeaux"   → [O=-4.2, B-DEP=5.1, I-DEP=-1.3, B-ARR=-3.8, I-ARR=-2.1]
                          →  B-DEPARTURE ✅
    token[10] "Strasbourg" → [O=-3.9, B-DEP=-2.4, I-DEP=-3.1, B-ARR=4.8, I-ARR=-1.7]
                           →  B-ARRIVAL ✅
```

2.3 Reconstruction des Entités

```
entities = {
    "DEPARTURE": "Bordeaux",    # tokens B-DEP reconstruit
    "ARRIVAL":   "Strasbourg"   # tokens B-ARR reconstruit
}
```

Post-processing : Filtrage des entités de moins de 2 caractères (aucune ici).

→ Départ = "Bordeaux", Arrivée = "Strasbourg"

Étape 3 : PATHFINDING — Calcul de l'Itinéraire

3.1 Normalisation des Noms de Villes

```
"Bordeaux"    → normalize → "bordeaux"
"Strasbourg"  → normalize → "strasbourg"
```

Recherche dans `gares-francaises.json` :

- "Bordeaux" → match exact → `"Bordeaux Saint-Jean"` (hub A, lat=44.83, lon=-0.56)
- "Strasbourg" → match partiel → `"Strasbourg"` (hub A, lat=48.58, lon=7.73)

3.2 Construction du Graphe (si non encore chargé)

```
Hubs A connectés ≤ 500 km entre eux:
  Bordeaux ↔ Paris     : 499 km ✓
  Bordeaux ↔ Lyon      : 417 km ✓
  Bordeaux ↔ Toulouse  : 213 km ✓
  Paris ↔ Strasbourg   : 447 km ✓
  Lyon ↔ Strasbourg    : 389 km ✓
  ...
```

3.3 Algorithme Dijkstra

```
File priorité initiale: [(0, "Bordeaux")]
Distances: {Bordeaux: 0, tous autres: ∞}

Itération 1: Extraire Bordeaux (dist=0)
  → Explorer voisins: Paris(499), Lyon(417), Toulouse(213)...
  → Distances: {Paris:499, Lyon:417, Toulouse:213, ...}

Itération 2: Extraire Toulouse (dist=213)
  → Explorer voisins depuis Toulouse
  → Aucun raccourci vers Strasbourg

Itération 3: Extraire Lyon (dist=417)
  → depuis Lyon → Strasbourg : 417 + 389 = 806 km
  → Distances: {Strasbourg: 806}

Itération 4: Extraire Paris (dist=499)
  → depuis Paris → Strasbourg : 499 + 447 = 946 km
  → 946 > 806, pas de mise à jour

→ Chemin retenu: Bordeaux → Lyon → Strasbourg (806 km)
```

3.4 Estimation du Temps de Trajet

```
Segment Bordeaux → Lyon (417 km):
  distance > 300 km → vitesse TGV = 280 km/h
  temps = 417 / 280 = 1.49 h

Segment Lyon → Strasbourg (389 km):
  distance > 300 km → vitesse TGV = 280 km/h
  temps = 389 / 280 = 1.39 h

Total = (1.49 + 1.39) × 1.10 (ajustement +10%) = 3.17 h
```

Étape 4 : Réponse Finale

4.1 Format JSON (API `/predict`)

```json
{
  "valid": true,
  "message": null,
  "departure": "Bordeaux",
  "arrival": "Strasbourg",
  "route": ["Bordeaux", "Lyon", "Strasbourg"],
```

```
  "route_distance": 806.0,
  "route_time": 3.17,
  "route_source": "graph",
  "route_algorithm": "dijkstra"
}
```

4.2 Format CSV (batch `/process_csv`)

Si entrée était `42,J'aimerais prendre le train de Bordeaux pour rejoindre Strasbourg`, la sortie est :

```
42,Bordeaux,Lyon,Strasbourg
```

4.3 Affichage Frontend

Le chatbot affiche :

```
🚂 Itinéraire trouvé !
Bordeaux → Lyon → Strasbourg
Distance : 806 km | Durée estimée : ~3h10
```

Récapitulatif du Flux

```
"J'aimerais prendre le train de Bordeaux pour rejoindre Strasbourg"
    |
    ▼ CLASSIFIER (CamemBERT)
    |  P(VALID) = 97.3%  → phrase valide ✅
    |
    ▼ NER (CamemBERT)
    |  DEPARTURE = "Bordeaux"
    |  ARRIVAL   = "Strasbourg"
    |
    ▼ PATHFINDING (Dijkstra sur graphe Haversine)
    |  Bordeaux → Lyon → Strasbourg
    |  806 km | ~3h10
    |
    ▼ RÉPONSE JSON
       { valid: true, route: ["Bordeaux","Lyon","Strasbourg"], ... }
```

Cas Limites Traités

Cas : Phrase invalide

```
Entrée: "Je veux manger une pizza"
Classifier: P(VALID) = 0.4% → INVALID
Réponse: { "valid": false, "message": "Phrase non valide" }
Sortie CSV: "X,INVALID"
```

Cas : Ville non trouvée dans le graphe

```
Entrée: "Je vais de Paris à New York"
Classifier: VALID
NER: DEPARTURE="Paris", ARRIVAL="New York"
```

```
Pathfinding: "New York" non trouvé → fallback / INVALID
Réponse: { "valid": false, "message": "Ville de destination non trouvée" }
```

Cas : Phrase ambiguë avec prénom

```
Entrée: "Mon ami Lyon va à Paris"
NER: Post-processing → "Lyon" = B-DEPARTURE, "Paris" = B-ARRIVAL
→ Le modèle peut se tromper (prénom "Lyon" pris comme ville)
→ C'est une limite connue, améliorable avec plus d'exemples négatifs
```

Document généré le 27/02/2026